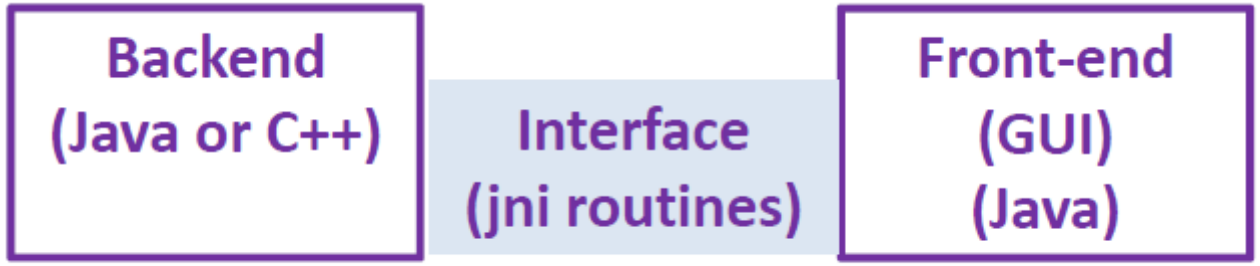
Introduction to Android Development

EC327 Introduction to Software Engineering

Overview

Your android apps can be written Java or C++ languages.

The Android SDK tools compile your code into an **APK, an Android package**, which is an archive file with an .apk suffix; this is what Android-powered devices use to install the app.



Environment and Tools

- Android Studio: Android integrated development environment
- Android NDK: Android Native Development Kit
  - If you would like to code the back-end in C++, most likely you need NDK
- Android SDK: Android SW Development Kit
  - Debugger, libraries, emulator, etc.

Resources

Android development resources:

- Build your first app: https://developer.android.com/training/basics/firstapp/index.html
- Android developer guides: https://developer.android.com/guide/

Java Resources

- Java for C++ programmers: https://www.cprogramming.com/tutorial/java/syntax-differences-java-c++.html
- Java tutorial: https://docs.oracle.com/javase/tutorial/index.html

Overall Structure

Android Studio project structure

- manifests: Contains the AndroidManifest.xml file.
- java: Contains the Java source code files, including JUnit test code.
- res: Contains all non-code resources, such as XML layouts, UI strings, and bitmap images.

UI elements

- Buttons, EditText, ListView…: all of these UI elements are declared as private members of the MainActivity class

```
    //Extends = inherits, implements means it interfaces with another class
public class MainActivity extends Activity implements OnClickListener {

        //Creating button objects, which extend (inherit) the View class
        private Button button10;
        private Button button15;
        private Button button20;
```

UI (Buttons)

1. Declare the button in xml file:

```
<Button
    android:id="@+id/tip20"
    android:text="@string/button_20"
    android:layout_width="match_parent"
    android:layout_height="wrap_content"
    android:layout_alignParentBottom="true"/>
```

main.xml

2. Linking the button declared in the xml file to the java file:

```
button20 = (Button) findViewById(R.id.tip20);
```

MainActivity.java

3. Set the OnClickListener function
   - the function that gets fed to the OnClickListener gets called when the user clicks on the button

```
button20.setOnClickListener(this);
```

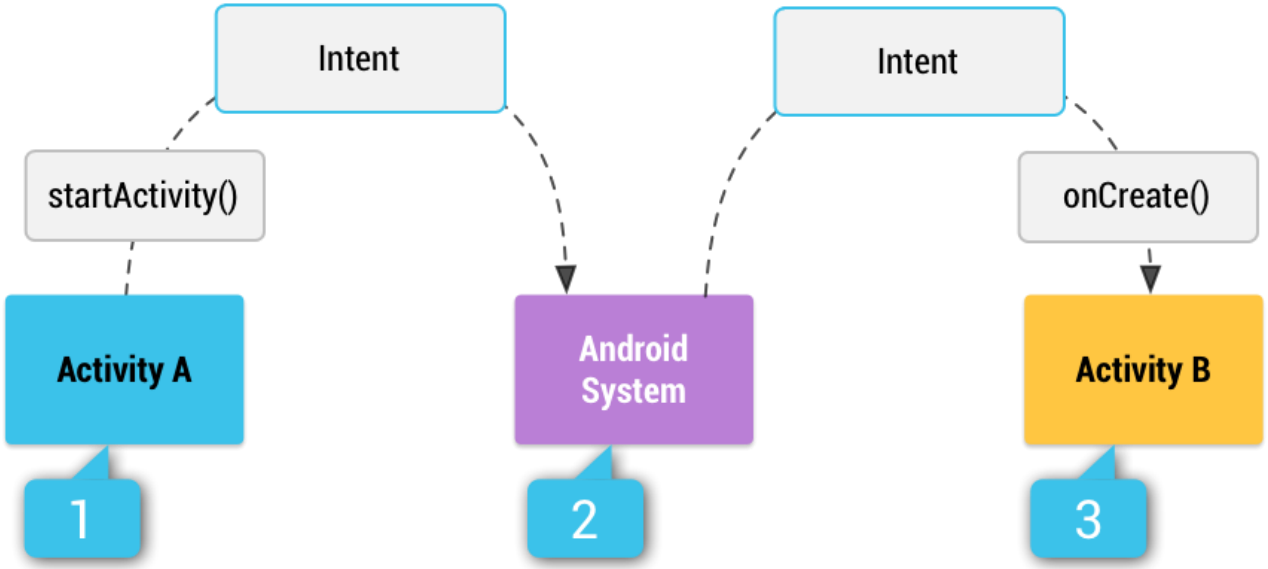
MainActivity.java

UX (Intents)

An intent is a messaging object used to request an action from another component. Used to:

- Start an activity
- Start a service
- Deliver a broadcast

Declare it in your Manifest.xml file:



Creating an intent (in MainActivity.java):

```
Intent resultActivity = new Intent(MainActivity.this, Result.class);
startActivity(resultActivity);
```

UX (Intents)

Creating an intent (in MainActivity.java):

```
Intent resultActivity = new Intent(MainActivity.this, Result.class);
startActivity(resultActivity);
```

Adding an intent to your AndroidManifest.xml:

```
    <activity android:name=".MainActivity">
        <intent-filter>
            <action android:name="android.intent.action.MAIN" />
            <category android:name="android.intent.category.LAUNCHER" />
        </intent-filter>
    </activity>
    <activity android:name="org.academicode.tipcalculator.Result"></activity>
```